

MACHINE LEARNING ASSIGNMENT REPORT

Student ID: 2551432

TASK 1: k-NN CLASSIFICATION

The k-Nearest Neighbours (k-NN) is a supervised learning algorithm used for classification and regression. It makes no assumption about the underlying distribution of the data, which makes it flexible and applicable in a wide range of problems.

The core idea behind it is to classify the unseen data point x , selecting the k nearest data points to new observation and assigns the class label that appears most frequently among those k neighbours (majority vote). The distance can be determined by Minkowski distance matrix with the parameter p .

Main hyperparameters

- K (number of neighbours) = The choice of K affects the model performance, with small k leading to overfitting and large k leading to underfitting.
- P (norm order) = The choice of p controls the distance matrix
 - $P = 1$: Manhattan distance
 - $P = 2$: Euclidean distance
 - $p \rightarrow \infty$: Chebyshev distance

In this task the personalised parameters were: $k = 6$ and $p = 6$

Train/Test split

A 70/30 train/test split was chosen, there were 398 samples in the training set and 171 samples in the test set. The ratio was selected because, k-NN is a distance-based method and relies on the training set, a large training set ensures that the neighborhood is populated with meaningful examples. With 171 test samples, the accuracy estimate has low variance. A small test set can yield unstable performance caused by sampling noise.

Calculation of $P(N | +)$

$P(N | +)$ is the probability that a patient is truly negative (benign) but the classifier predicts that it is positive (Malignant).

Using the notation from the confusion matrix:

- FP = negative sample incorrectly predicted as positive.
- TP = positive sample correctly predicted as positive.

To find the probability we need to apply Bayes' theorem in the confusion matrix counts:

$$P(N|+) = \frac{FP}{TP + FP}$$

According to my output:

FP =8

TP=100

$$P(N|+) = \frac{8}{100 + 8} \approx 0.0741$$

There is 7.41% probability that patient is truly negative but classifier predicts it positive.

TASK 2

Principal Component Analysis (PCA):

Principal Component Analysis (PCA) is a dimensionality reduction technique, it tells us the direction in which most of the variance occur.

Outline algorithm:

- Let we have our D- column by N-row data set X. To ensure that PCA capture the true direction of the variation rather than being bias we centered the data (subtracting the mean from each feature)

$$X \in \mathbb{R}^{N \times P}$$

$$X_{centered} = X - \mu$$

- Compute the covariance matrix and solve for the eigenvalues and eigenvectors:

$$S = \frac{1}{N} X^T X$$

$$Sv = \lambda v$$

- Then we have to form a M- column by D-row matrix (B) from eigenvectors that define the principal subspace. we can project and define the Z as follows:

$$Z = XBB^T$$

- If M = D, then we can say that B = X. Otherwise $M \leq D$ we can say Z is a projection of X in a lower dimensional space

Results:

The results are comparable. The accuracy barely changes , the confusion matrix are also almost identical, one false negative in task 1 becomes true positive in task 2 but two true negative becomes false positive. I recommend using just nc=3 principal components because the three component captures 88.96% of the total variance and model perform equally well, in fact little bit better than using all 5 components. The two discarded component have only 11.04% of the variance and maybe it is noise rather useful information.

If I talk about the probability which we have calculated in task 1 $P(N|+)$, the chance of predicting false positive is decreased to 6.48% which means it decreased the false alarm and increase the true positive which means PCA marginally improves the detection of the true malignant case. If we calculate the probability again using PCA then it would be

So, the 3 component model is recommended as it is simpler, less noisy and equally accurate.

TASK 3:

Singular Value Decomposition

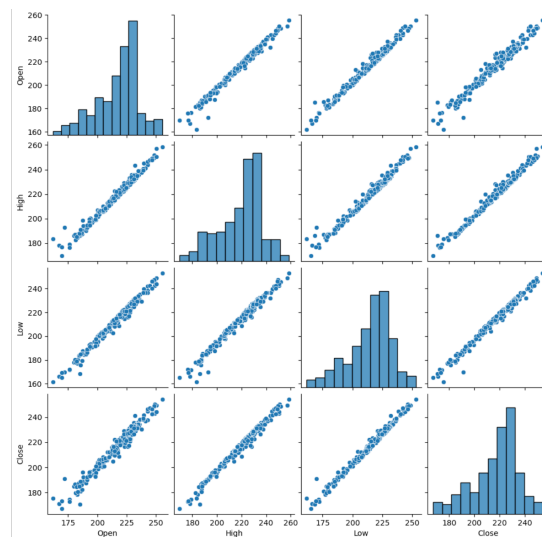
Singular Value Decomposition (SVD) is a matrix factorization technique that decomposes any matrix $X(mn)$ into three matrices:

$$X = U \Sigma V^T$$

- $U(m \times m)$: Orthogonal matrix, columns are the left singular vectors.
- $\Sigma(m \times n)$: Diagonal matrix of non-negative singular values $\sigma_1 \geq \sigma_2 \geq \dots \geq 0$
- $V^T(n \times n)$: Orthogonal matrix row are the right singular vectors.

Pair plot of dfth:

The pair plot of dfth shows Open, High, Low and Close plotted against each other, with the histogram on the diagonal. Every scatterplot shows an almost perfectly straight diagonal line and every pair is almost perfectly linearly correlated maybe because all four prices on the trading day are from same Amazon stock price. The price range is from around \$160 to \$255 over the 231 days.



The histogram on the diagonal all share a very similar distribution shape (right skewed) most of the trading days have price between \$180 to \$230, with fewer days at the extremes.

Since all four variables are almost the same, the data does not really have four dimensions. So I would expect just 1 dominant independent component in this data. There might be very small second source of variation, which would be the difference between the High and Low but this is much smaller.

Rank of the dataset :

The matrix X_{train} (231×4) has full rank 4, all four singular values are non-zero. However, $\sigma_1 = 6625.81$ is 172 times larger than $\sigma_2 = 38.54$. The matrix is technically full ranked but effectively rank 1 because one component dominates so heavily.

SVD transformation of X_train to Kc :

To get compressed version of training data , first we have performed SVD on training data set which gives us three matrices U , S and VT, where :

$$X_{train} = U \Sigma V^T$$

To compress the training data in to c component, we kept only the first c columns of U, the first c singular values from S, and first c rows of VT. The compressed training matrix Kc is then :

$$Kc = U_c \Sigma_c V_c^T$$

For c=1 this means we have only take first singular value $\sigma_1 = 6625.81$ and its corresponding singular vectors. Since only one component is kept, every row of Kc is just a scaled version of the same vector. This means all 231 data points get mapped onto a single straight line.

For c = 2, we kept first two component, so the data now lie in a 2D plane rather than a line. The reconstruction error from my notebook confirms that the error at c = 1 is 51.48 and drops to 34.13 at c = 2 , showing that adding a second component improves the approximation .

SVD transformation of X_test to Qc :

For the test data we cant just run a new SVD because that would use the information from the test set to define the transformation which is a cheating, so instead we have used the VT matrix that was already learned from training data to project test data into same subspace as Kc.

The transformation is used:

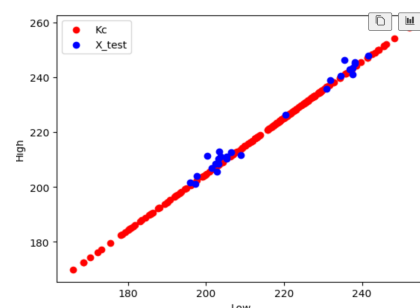
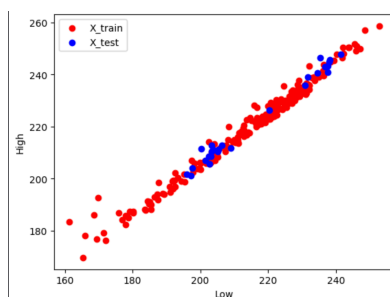
$$Qc = X_{test} \times V_c \times V_c^T$$

It will first project the test data into c dimensional subspace defined by the first row of VT and then reconstruct it back into the 4D feature space . this way, Qc and Kc are in the same subspace and can be directly compared on the same scatter plot.

For c = 1 : how does Plot 2 (Kc vs X_test) differ from Plot 1 (X_train vs X_test) :

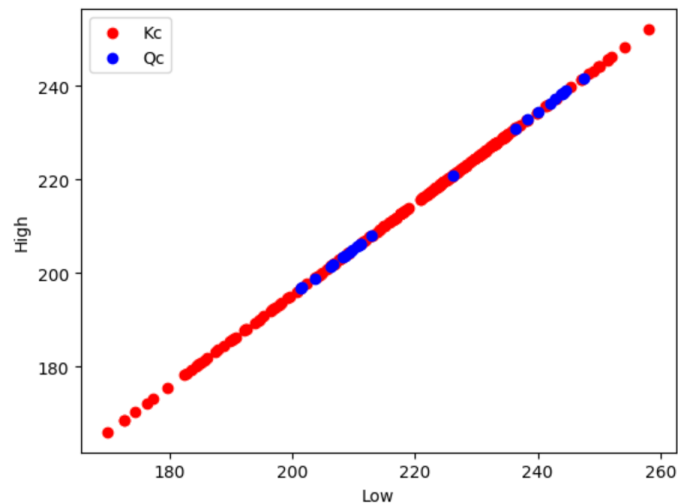
In plot 1 both X_train(red) and X_test(blue) show a natural scatter of points. There is visible spread around the trend, specially in the lower region (around 160-180) where few red points are scattered loosely. The blue points are sitting in the upper portion of the plot (200-250) and blending naturally with the red line.

In plot 2 the X_train is same as before but the red point(Kc) are now in a perfectly straight line. The points which were loosely scatter in plot 1 are disappeared. This is because every training point is projected onto a single directional the X_test still have there original scatter.



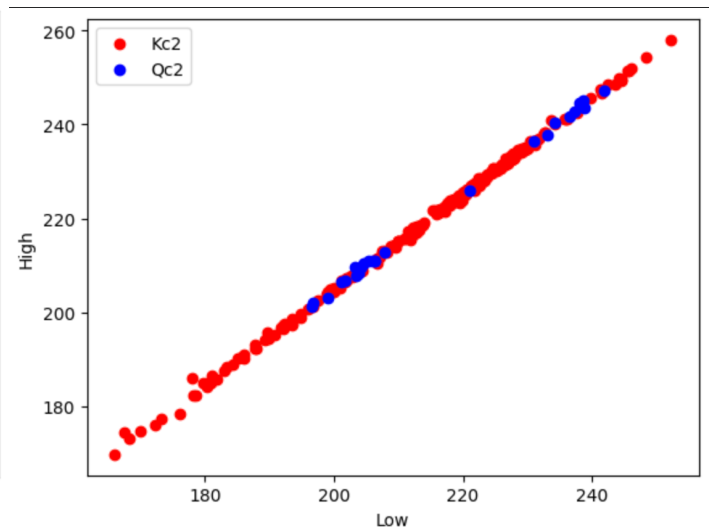
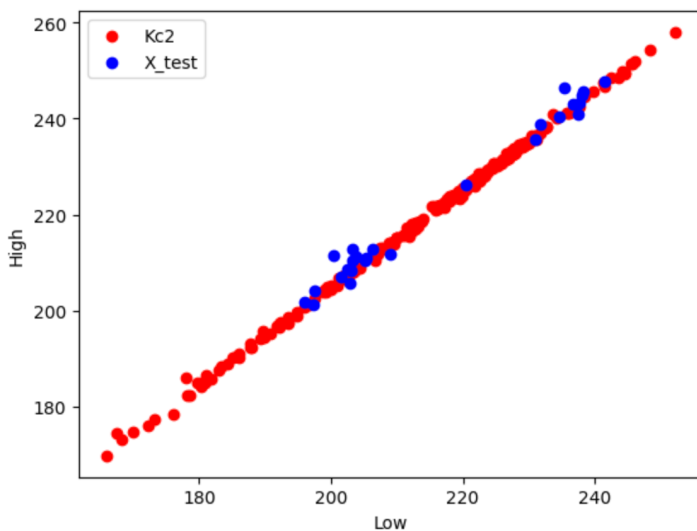
For $c = 1$: how does Plot 3 (Kc vs Qc) differ from the first two :

In plot 1 and 2, X_{test} has scattered point but Kc has collapsed onto a single straight line. But in plot 3, both Kc and Qc lie on the same line because Qc has been projected onto the same subspace as Kc . The only difference is the Qc sit higher then Kc because the test data is from January 2026 (230-245) has higher price than the training data Feb-Dec (195-225). Plot 3 therefore the best comparison with the same structure but different price level.



For $c=2$: how do the plots changes :

Plot 2 (Kc_2 vs X_{test}) looks very different from $c=1$ version, here the red Kc_2 point still follow the same diagonal direction but now a little bit thickness and spread around the line. The points are no longer exactly on the top of each other. This shows the second component has recovered some of the natural spread in the data that $c = 1$ threw away.



Plot 3 (Kc_2 vs Qc_2) both lies in the same 2D subspace. The alignment is much better than $c=1$, they are overlapping in structure rather than just being two co linear lines.

Moving from $c=1$ to $c=2$ recover the spread between High and Low, making the compressed data much reliable and useful representation of the original data. The first singular value (6625.81) is 172 times larger than second singular value (38.54) but the second singular value restores the shape of the data even though it is small.